

Concurrency within an Operating System and Its Mechanisms

Operating Systems -
SKU 6.2.1

Overview

- ▶ • What is concurrency?
- ▶ • Concurrency challenges
- ▶ • OS mechanisms for concurrency
- ▶ • Scheduling, dispatch, synchronization
- ▶ • Memory and security considerations

What Is Concurrency?

- ▶ • Ability of an OS to manage multiple tasks at the same time
- ▶ • Tasks may not run simultaneously (single CPU) but appear to
- ▶ • Essential for multitasking and performance
- ▶ • Managed through scheduling and process control

Concurrency Challenges

- ▶ • Race conditions
- ▶ • Deadlocks
- ▶ • Starvation
- ▶ • Data inconsistency
- ▶ • Resource contention

Processes and Threads

- ▶ • Process: independent executing program
- ▶ • Thread: smallest unit of execution
- ▶ • Threads share memory, processes do not
- ▶ • Multithreading enables fine-grained concurrency

Scheduling

- ▶ • Determines which process/thread runs next
- ▶ • Common algorithms:
 - ▶ - FCFS (First-Come, First-Served)
 - ▶ - Round Robin
 - ▶ - Priority Scheduling
 - ▶ - Multilevel Queue Scheduling

Dispatch Mechanism

- ▶ • Dispatcher starts execution of selected process
- ▶ • Performs context switching
- ▶ • Switches CPU mode and memory mapping
- ▶ • Ensures smooth transition between tasks

Synchronization Mechanisms

- ▶ • Prevents race conditions and ensures safe access
- ▶ • Common mechanisms:
 - ▶ - Mutex locks
 - ▶ - Semaphores
 - ▶ - Monitors
 - ▶ - Condition variables

Deadlocks

- ▶ • Occurs when processes wait indefinitely for resources
- ▶ • Four necessary conditions:
 - ▶ - Mutual exclusion
 - ▶ - Hold and wait
 - ▶ - No preemption
 - ▶ - Circular wait
- ▶ • OS strategies: prevention, avoidance, detection, recovery

Memory Management & Concurrency

- ▶ • OS isolates processes via virtual memory
- ▶ • Prevents accidental interference between processes
- ▶ • Paging and segmentation support concurrent execution
- ▶ • Shared memory requires synchronization

Concurrency and Security

- ▶ • Concurrency errors can cause vulnerabilities
- ▶ • Privilege separation prevents escalation
- ▶ • Kernel ensures safe context switching
- ▶ • Access control protects shared resources

Summary

- ▶ • Concurrency enables multitasking and efficient CPU use
- ▶ • OS uses scheduling, dispatch, and synchronization mechanisms
- ▶ • Memory management and security are essential to safe execution
- ▶ • Core concept across modern operating system design

Runtime Problems in Concurrent Task Execution

Operating Systems -
SKU 6.2.2

Introduction to Concurrency

- ▶ Concurrency = multiple tasks executing overlapping in time
 - ▶ OS coordinates task interactions
 - ▶ Concurrency improves throughput but introduces runtime issues

Key Runtime Problems

- ▶ Race conditions
 - ▶ Deadlocks
 - ▶ Livelocks
 - ▶ Starvation
 - ▶ Priority inversion

Race Conditions

- ▶ Multiple tasks access shared data simultaneously
 - ▶ Incorrect/inconsistent results if not synchronized
 - ▶ Occurs in: counters, buffers, shared variables

Deadlocks

- ▶ Processes wait indefinitely for resources
 - ▶ All tasks involved are blocked
 - ▶ Requires: mutual exclusion, hold/wait, no preemption, circular wait

Livelocks

- ▶ Processes continuously change state but make no progress
 - ▶ Similar to deadlocks but actively 'spinning'
 - ▶ Example: two processes constantly releasing/acquiring resources

Starvation

- ▶ Task never receives needed CPU or resources
 - ▶ Caused by unfair scheduling or resource monopolization
 - ▶ Low-priority tasks most vulnerable

Priority Inversion

- ▶ Low-priority task holds resource needed by high-priority task
 - ▶ Medium-priority tasks delay low-priority release
 - ▶ Solution: priority inheritance

Scheduling-Related Runtime Issues

- ▶ Poor scheduling increases context-switch overhead
 - ▶ Cache thrashing when many tasks interchange
 - ▶ Unfair scheduling leads to long wait times

Synchronization Problems

- ▶ Incorrect locking → deadlocks or race conditions
 - ▶ Over-synchronization → performance bottlenecks
 - ▶ Under-synchronization → data corruption

Memory Management Conflicts

- ▶ Concurrent tasks compete for RAM
 - ▶ High concurrency → more page faults
 - ▶ Thrashing possible when working sets exceed memory

Resource Contention

- ▶ CPU, I/O, memory, and network are finite
 - ▶ High concurrency causes queues and delays
 - ▶ OS must mediate fair and efficient access

Strategies to Mitigate Issues

- ▶ Synchronization primitives (mutexes, semaphores, monitors)
 - ▶ Deadlock prevention/avoidance algorithms
 - ▶ Fair and priority-aware scheduling
 - ▶ Working-set model to reduce memory thrashing

Summary

- ▶ Concurrency introduces powerful capabilities but also runtime hazards
 - ▶ OS mechanisms mitigate race conditions, deadlocks, starvation, etc.
 - ▶ Proper design and scheduling ensure safe and efficient execution

Task States & Concurrency Issues

Operating Systems -
SKU 6.2.3

Overview

- ▶ • States of a task
- ▶ • Concurrent operation of tasks
- ▶ • Runtime problems in concurrency
- ▶ • OS mechanisms for handling concurrency

What is a Task?

- ▶ • A task is an executing program (process or thread)
- ▶ • Contains execution context (registers, memory, program counter)
- ▶ • Managed by the operating system
- ▶ • Can exist concurrently with many others

Task Lifecycle States

- ▶ • New - Task is being created
- ▶ • Ready - Awaiting CPU availability
- ▶ • Running - Executing on the CPU
- ▶ • Blocked - Waiting for I/O or an event
- ▶ • Terminated - Finished executing

Why Multiple Task States?

- ▶ • Allows OS to manage many tasks simultaneously
- ▶ • Supports efficient CPU utilization
- ▶ • Helps avoid interference among tasks
- ▶ • Enables smooth scheduling and resource management

Concurrency in Operating Systems

- ▶ • Multiple tasks appear to run simultaneously
- ▶ • Achieved through CPU scheduling
- ▶ • Tasks may share resources (memory, I/O, files)
- ▶ • Leads to potential runtime conflicts

Runtime Problems in Concurrent Task Execution

- ▶ • Race conditions
- ▶ • Deadlocks
- ▶ • Starvation
- ▶ • Priority inversion
- ▶ • Data inconsistency

Race Conditions

- ▶ • Occur when tasks access shared data improperly
- ▶ • Outcome depends on unpredictable execution order
- ▶ • Can cause incorrect or inconsistent results
- ▶ • Requires locks, mutexes, or semaphores

Deadlocks

- ▶ • Tasks wait indefinitely for resources held by each other
- ▶ • Example: A waits for B, B waits for A
- ▶ • OS solutions: Prevention, Avoidance, Detection & Recovery

Starvation & Priority Inversion

- ▶ • Starvation: Task is denied resources for too long
- ▶ • Priority inversion: Low-priority task blocks high-priority task
- ▶ • OS uses fairness algorithms and priority inheritance

Mechanisms for Managing Concurrency

- ▶ • Scheduling algorithms (Round Robin, Priority, etc.)
- ▶ • Synchronization tools (mutexes, semaphores, monitors)
- ▶ • Resource allocation and protection
- ▶ • Memory isolation and access control

Conclusion

- ▶ • Tasks transition through multiple states during execution
- ▶ • Concurrency introduces challenges requiring OS intervention
- ▶ • Proper synchronization ensures safety and efficiency
- ▶ • Scheduling and resource management are essential for performance

Techniques for Achieving Synchronization in an Operating System

Operating Systems -
SKU 6.2.4

Overview

- ▶ • Need for synchronization
- ▶ • Critical section problem
- ▶ • Hardware-based techniques
- ▶ • OS-level mechanisms
- ▶ • High-level synchronization tools

Why Synchronization is Needed

- ▶ • Prevent race conditions
- ▶ • Ensure data consistency
- ▶ • Coordinate access to shared resources
- ▶ • Enable safe concurrent execution

Critical Section Problem

- ▶ A solution must satisfy:
- ▶ • Mutual exclusion - Only one process in critical section
- ▶ • Progress - No process waits unnecessarily
- ▶ • Bounded waiting - Prevent starvation

Hardware Techniques

- ▶ • Interrupt disabling
- ▶ • Test-and-Set instruction
- ▶ • Swap instruction
- ▶ • Compare-and-Swap (CAS)
- ▶ • Useful for low-level locking mechanisms

Mutex Locks

- ▶ • Ensures only one thread enters a critical section
- ▶ • Simple lock() and unlock() operations
- ▶ • May cause busy waiting (spinlocks)

Semaphores

- ▶ • Counting and binary semaphores
- ▶ • wait() and signal() operations
- ▶ • Avoid busy waiting
- ▶ • Useful for signaling between tasks

Monitors

- ▶ • High-level synchronization construct
- ▶ • Allows only one process inside at a time
- ▶ • Uses condition variables for waiting
- ▶ • Widely used in modern languages (Java, C#, Python)

Spinlocks

- ▶ • Threads continuously check lock availability
- ▶ • Useful in multiprocessor systems
- ▶ • Low overhead for short critical sections

Barriers

- ▶ • Used for process synchronization in parallel tasks
- ▶ • All tasks must reach a point before any can proceed
- ▶ • Common in scientific and multi-threaded applications

Readers-Writers Synchronization

- ▶ • Allows multiple readers but exclusive writer access
- ▶ • Solutions prevent starvation of readers or writers
- ▶ • Implemented with semaphores or read-write locks

Summary

- ▶ • Synchronization ensures safe concurrent execution
- ▶ • Hardware and software techniques work together
- ▶ • Semaphores, mutexes, monitors widely used
- ▶ • Proper synchronization prevents race conditions

State and Transition Diagrams

Operating Systems -
SKU 6.2.5

What Are State and Transition Diagrams?

- ▶ Graphical models representing states of a system
 - ▶ Show transitions caused by events
 - ▶ Useful for analyzing OS process behavior

States in an Operating System

- ▶ New
 - ▶ Ready
 - ▶ Running
 - ▶ Waiting/Blocked
 - ▶ Terminated

Transitions in OS

- ▶ Dispatch: Ready → Running
 - ▶ Timeout: Running → Ready
 - ▶ I/O Wait: Running → Blocked
 - ▶ I/O Completion: Blocked → Ready
 - ▶ Exit: Running → Terminated

Concurrency and State Diagrams

- ▶ Model simultaneous processes
 - ▶ Detect deadlocks and race conditions
 - ▶ Represent synchronization points

Scheduling & Dispatch

- ▶ State transitions depend on scheduler decisions
 - ▶ Preemptive vs non-preemptive transitions
 - ▶ Dispatcher moves processes into Running state

Memory Management Impact

- ▶ Page faults cause transitions to waiting state
 - ▶ Swapping affects ready/waiting state placement
 - ▶ Memory availability influences scheduling

Security Considerations

- ▶ State transitions must enforce permissions
 - ▶ Prevent unauthorized state changes
 - ▶ Secure handling of interrupts & traps

Applications of State Diagrams

- ▶ System debugging
 - ▶ Performance analysis
 - ▶ Formal verification of OS correctness

Processes, Threads, Scheduling Algorithms

Operating Systems -
SKU 6.2.6

What Are Processes?

- ▶ Independent units of execution
 - ▶ Contain code, data, and OS-managed state
 - ▶ Scheduled by the OS

What Are Threads?

- ▶ Lightweight units within a process
 - ▶ Share address space and resources
 - ▶ Enable parallelism & responsiveness

Processes vs Threads

- ▶ Processes: isolated, heavy context switching
 - ▶ Threads: faster context switching
 - ▶ Threads enable concurrency within applications

Scheduling Algorithms

- ▶ First-Come, First-Served (FCFS)
 - ▶ Shortest Job First (SJF)
 - ▶ Round Robin (RR)
 - ▶ Priority Scheduling
 - ▶ Multilevel Queue & Feedback Queue

Preemptive vs Non-Preemptive Scheduling

- ▶ Preemptive: OS interrupts running task
 - ▶ Non-preemptive: Tasks run until completion or block
 - ▶ Impact on response time and throughput

Relationship to Application Domains

- ▶ Real-time systems require predictable scheduling
 - ▶ Interactive applications favor short response time
 - ▶ Batch processing benefits from throughput-optimized algorithms

Concurrency and Application Behavior

- ▶ Threads support parallel computations
 - ▶ Concurrency improves UI responsiveness
 - ▶ OS manages synchronization & race condition prevention

Impact of Scheduling on Performance

- ▶ Algorithm choice impacts CPU utilization
 - ▶ Thread-heavy workloads need efficient context switching
 - ▶ Balancing fairness vs throughput

Security and Process/Thread Management

- ▶ Process isolation prevents unauthorized access
 - ▶ Thread interactions must be controlled
 - ▶ OS enforces permission and resource limits

Summary

- ▶ Processes and threads form the core of OS concurrency
 - ▶ Scheduling algorithms determine performance characteristics
 - ▶ Application needs guide OS-level scheduling strategies

Memory Hierarchy and Cost- Performance Tradeoffs

Operating Systems -
SKU 6.2.7

What Is Memory Hierarchy?

- ▶ Organized layers of storage systems
 - ▶ Designed to balance speed, cost, and capacity
 - ▶ OS manages data placement across levels

Levels of Memory Hierarchy

- ▶ Registers - fastest, smallest, most expensive
 - ▶ Cache (L1, L2, L3) - near-CPU, fast access
 - ▶ Main Memory (RAM) - moderate speed & cost
 - ▶ Secondary Storage - SSD/HDD, slower, cheap
 - ▶ Tertiary Storage - archival, very cheap, very slow

Principles Behind the Hierarchy

- ▶ Temporal locality - recent data reused
 - ▶ Spatial locality - nearby data likely needed
 - ▶ Hierarchy uses locality to speed computation

Cost-Performance Tradeoffs

- ▶ Fast memory is expensive and small
 - ▶ Cheap memory is slow but large
 - ▶ Hierarchy combines multiple technologies
 - ▶ OS decides data movement (e.g., paging)

Cache and OS Behavior

- ▶ OS scheduling can influence cache usage
 - ▶ Context switches may cause cache pollution
 - ▶ Cache-aware algorithms improve performance

Memory Management & Hierarchy

- ▶ Virtual memory extends logical space
 - ▶ Page faults move data from secondary to main memory
 - ▶ OS must balance performance vs memory pressure

Tradeoff Examples

- ▶ Using more cache → higher cost but lower latency
 - ▶ Large RAM → fewer page faults but expensive
 - ▶ SSD vs HDD → speed vs storage capacity

Impact on Application Performance

- ▶ Compute-heavy apps rely on fast caches
 - ▶ Database systems benefit from large RAM
 - ▶ Archival applications rely on slow/cheap storage

Modern Trends

- ▶ Non-volatile memory (NVM) closing the gap
 - ▶ Hybrid memory systems common
 - ▶ OS must adapt policies for new memory tech

Summary

- ▶ Memory hierarchy balances performance and cost
 - ▶ OS plays key role in managing data movement
 - ▶ Tradeoffs affect application efficiency

Virtual Memory, Memory Allocation, Cache Memory, and Thrashing

Operating Systems -
SKU 6.2.8

Principles of Virtual Memory

- ▶ Provides illusion of large memory space
 - ▶ Uses paging to map virtual → physical memory
 - ▶ Allows execution of processes larger than RAM
 - ▶ Separates logical memory from physical memory

Benefits of Virtual Memory

- ▶ Increased multiprogramming
 - ▶ Isolation between processes
 - ▶ Efficient and safe memory use
 - ▶ Simplifies programming model

Tasks in Memory Allocation

- ▶ Assign memory blocks to processes
 - ▶ Reuse freed memory
 - ▶ Prevent fragmentation
 - ▶ Techniques: fixed partitioning, dynamic allocation

Dynamic Memory Allocation Methods

- ▶ First Fit - first available block
 - ▶ Best Fit - smallest suitable block
 - ▶ Worst Fit - largest available block
 - ▶ Buddy System - efficient block splitting and merging

Cache Memory Overview

- ▶ Small, fast memory close to CPU
 - ▶ Stores frequently used instructions/data
 - ▶ Levels: L1 (fastest), L2, L3
 - ▶ Reduces average memory access time

Cache Mapping Techniques

- ▶ Direct-mapped cache
 - ▶ Fully associative cache
 - ▶ Set-associative cache
 - ▶ Tradeoffs: speed vs complexity

Paging and Performance

- ▶ Page table manages virtual-to-physical mapping
 - ▶ TLB caches recent page translations
 - ▶ Reduces page lookup overhead

What Is Thrashing?

- ▶ Occurs when excessive paging takes place
 - ▶ CPU spends more time swapping pages than executing
 - ▶ Causes drastic performance drop

Causes of Thrashing

- ▶ Insufficient physical memory
 - ▶ High degree of multiprogramming
 - ▶ Poor locality of reference
 - ▶ Large working sets exceeding RAM

Preventing Thrashing

- ▶ Working Set Model
 - ▶ Page Fault Frequency (PFF) control
 - ▶ Reducing multiprogramming level
 - ▶ Increasing RAM or optimizing workloads

Summary

- ▶ Virtual memory improves flexibility and efficiency
 - ▶ Memory allocation affects performance and fragmentation
 - ▶ Cache memory speeds up CPU access
 - ▶ Thrashing must be avoided for stable system performance

Protection and Security in an Operating System

Operating Systems -
SKU 6.2.9

What Is Protection in an OS?

- ▶ Mechanisms for controlling access to system resources
 - ▶ Ensures programs and users do not interfere with one another
 - ▶ Policies define what is allowed; mechanisms enforce rules

What Is Security in an OS?

- ▶ Defends system against internal/external threats
 - ▶ Protects confidentiality, integrity, and availability
 - ▶ Addresses malware, unauthorized access, and attacks

Goals of OS Protection

- ▶ Prevent accidental or malicious misuse
 - ▶ Control access to memory, files, I/O devices
 - ▶ Maintain stable, predictable system behavior

Mechanisms for Protection

- ▶ Access Control Lists (ACLs)
 - ▶ Capabilities and tokens
 - ▶ User modes: kernel mode vs user mode
 - ▶ Memory protection (segmentation, paging)

Security Mechanisms in OS

- ▶ Authentication (passwords, biometrics, tokens)
 - ▶ Authorization (role-based or rule-based access)
 - ▶ Encryption of data and communication
 - ▶ Auditing and logging to track activity

Protection Domains

- ▶ Define bounds of authority for processes
 - ▶ User roles determine allowed operations
 - ▶ Dynamic switching between domains

Threats and Vulnerabilities

- ▶ Malware: viruses, worms, trojans
 - ▶ Privilege escalation
 - ▶ Buffer overflow attacks
 - ▶ Social engineering compromises authentication

OS Features Supporting Security

- ▶ Process isolation
 - ▶ Secure file permissions
 - ▶ Sandboxing untrusted applications
 - ▶ Firewalls and secure network stack

Limitations of OS Protection

- ▶ Cannot protect against hardware failure
 - ▶ Weak authentication policies lead to breaches
 - ▶ Insider threats remain difficult to mitigate
 - ▶ Misconfiguration reduces effectiveness

Limitations of OS Security

- ▶ Zero-day vulnerabilities may bypass controls
 - ▶ Resource limits may hinder security checks
 - ▶ Performance overhead from encryption & monitoring
 - ▶ Dependent on user behavior and update practices

Improving OS Protection & Security

- ▶ Regular patching and updates
 - ▶ Least privilege principle
 - ▶ Secure configuration and auditing
 - ▶ Use of virtualization and containerization

Summary

- ▶ Protection focuses on controlled resource access
 - ▶ Security protects system from threats
 - ▶ Both depend on OS features, configuration, and user practices
 - ▶ Limitations must be managed with layered defense

Mechanisms in an OS to Control Access to Resources

Operating Systems -
SKU 6.2.10

Introduction to Access Control

- ▶ OS controls which users/processes can access system resources
 - ▶ Ensures safe, reliable, controlled usage
 - ▶ Prevents unauthorized operations

Core OS Access Control Mechanisms

- ▶ Authentication - verifies identity
 - ▶ Authorization - determines allowed actions
 - ▶ Accounting - logs resource usage

Authentication Methods

- ▶ Passwords and PINs
 - ▶ Biometric verification
 - ▶ Smart cards and security tokens
 - ▶ Multi-factor authentication

Authorization Approaches

- ▶ Role-Based Access Control (RBAC)
 - ▶ Discretionary Access Control (DAC)
 - ▶ Mandatory Access Control (MAC)
 - ▶ Rule-Based Access Control

Access Control Lists (ACLs)

- ▶ List permissions for users/groups
 - ▶ Applied to files, directories, devices
 - ▶ Allows fine-grained permission control

Capabilities

- ▶ Token or key granting specific privileges
 - ▶ Attached to processes
 - ▶ More secure than ACLs against confused-deputy problems

Memory Protection Techniques

- ▶ Segmentation and paging isolate processes
 - ▶ Prevents access to unauthorized memory regions
 - ▶ Protects against buffer overflows

Protection Rings

- ▶ Hierarchical privilege levels
 - ▶ Kernel in highest privilege (Ring 0)
 - ▶ Applications run in lower privilege (Ring 3)
 - ▶ Prevents direct hardware access

File System Permissions

- ▶ Read/Write/Execute attributes
 - ▶ User, Group, and Others permissions
 - ▶ Used in UNIX, Linux, Windows file systems

I/O and Device Access Control

- ▶ Device drivers enforce permissions
 - ▶ OS mediates all hardware access
 - ▶ Prevents unauthorized device manipulation

Virtualization-Based Isolation

- ▶ Containers isolate processes
 - ▶ Virtual machines provide hardware-level isolation
 - ▶ Useful for security and controlled resource sharing

Summary

- ▶ OS uses multiple mechanisms to enforce access control
 - ▶ Authentication + authorization form core structure
 - ▶ Isolation and protection techniques ensure safe operation

System Administration Tasks According to Security Policies

Operating Systems -
SKU 6.2.11

Introduction to System Administration & Security

- ▶ System administration ensures secure, stable system operation
 - ▶ Security policies guide admin responsibilities
 - ▶ OS enforces policy compliance

Security Policies Overview

- ▶ Define rules for system access, usage, and behavior
 - ▶ Ensure confidentiality, integrity, and availability
 - ▶ Administrators enforce and maintain policies

User and Account Management

- ▶ Create, modify, and remove user accounts
 - ▶ Assign roles and permissions
 - ▶ Enforce password policies and authentication requirements

Access Control Management

- ▶ Manage file and directory permissions
 - ▶ Configure Access Control Lists (ACLs)
 - ▶ Implement least privilege principle

Patch and Update Management

- ▶ Apply OS and application patches
 - ▶ Monitor for vulnerabilities
 - ▶ Automate updates to reduce security risk

Monitoring and Logging

- ▶ Enable system logs and audit trails
 - ▶ Monitor login attempts and resource usage
 - ▶ Detect unusual or unauthorized activities

Backup and Recovery Tasks

- ▶ Schedule periodic backups
 - ▶ Verify backup integrity
 - ▶ Implement disaster recovery procedures

Network Security Administration

- ▶ Configure firewalls and access filters
 - ▶ Manage secure network protocols
 - ▶ Monitor network traffic for threats

Malware Prevention & Response

- ▶ Deploy antivirus and anti-malware tools
 - ▶ Perform regular scans
 - ▶ Respond to infections and isolate compromised systems

Policy Compliance & Documentation

- ▶ Maintain documentation for security procedures
 - ▶ Ensure systems comply with corporate or institutional policies
 - ▶ Conduct security audits and reviews

Incident Response Procedures

- ▶ Identify and contain security breaches
 - ▶ Document incidents for analysis
 - ▶ Implement corrective measures to prevent recurrence

Summary

- ▶ System administrators enforce security policies through routine tasks
 - ▶ Proper administration ensures reliability and protection
 - ▶ Security policies guide access, updates, monitoring, and response